

Docker + Git Portfolio Plan (1 Hour/Day) How to Use This Guide **Goal:** Not just “learn Docker”, but build a **visible portfolio** on GitHub with real, reproducible work (labs + a mini CI/CD project).

Where to run the commands:

- **Local machine with Linux (Ubuntu)** — recommended for realism.
- **Docker Desktop on Windows** — use the integrated terminal or WSL2 (Ubuntu).
- You **do not** run Docker in “Git terminal” or GitHub UI. You work **locally** (Ubuntu / WSL / Docker Desktop), then push your work to GitHub.

Workflow:

1. Work locally (Ubuntu / WSL / Docker Desktop terminal).
2. Save files in a project folder (e.g. docker-lab).
3. Use Git locally to commit the changes.
4. Push everything to a GitHub repository — this is your portfolio.

Building a Portfolio in Git **Step 1 — Create the Main Repo**

- On GitHub, create a public repo: docker-portfolio.
- Locally:
mkdir docker-portfolio
cd docker-portfolio
git init
git remote add origin https://github.com/USERNAME/docker-portfolio.git

Step 2 — Structure the Portfolio

Recommended folder layout:

```
docker-portfolio/  
README.md  
LOG.md  
basics/  
day1-basic-image/  
day2-containers/  
day3-images/  
day4-volumes/  
day5-networking/  
day6-compose/  
day7-incident/  
projects/  
ci-cd-demo/
```

- **README.md** — High-level description of what this repo shows (Docker learning + mini CI/CD project).
- **LOG.md** — Daily log: one line per day of what you built/fixed.
- **basics/** — All 7 days of Docker exercises.
- **projects/ci-cd-demo/** — Your mini portfolio project (CI/CD + containers).

Step 3 — Git Routine (Every Day)

After you finish your 1h session:

```
git add .  
git commit -m "Day 3: Image optimization with Alpine"  
git push origin main
```

Step 4 — Branches for Bigger Work

For the CI/CD project or refactors:

```
git checkout -b feature/ci-cd-demo  
# work...  
git add .  
git commit -m "Add basic CI pipeline"
```

`git push --set-upstream origin feature/ci-cd-demo`

Then do the merge on GitHub via Pull Request — looks professional and shows you know basic Git workflow.

Mini Project: CI/CD + Containers (Portfolio Piece) **Objective:** Build a simple app + container + CI pipeline that:

- Builds a Docker image
- Runs tests (even basic ones)
- Pushes the image to a registry (optional, can be simulated)

Suggested stack:

- Simple app: small Node.js or Python web app (REST or “Hello from Docker”).
- Docker: One Dockerfile building and running the app.
- Compose (optional): App + simple service (e.g. fake DB).
- CI: GitHub Actions workflow.

Project structure (inside projects/ci-cd-demo/):

ci-cd-demo/
app/
app.py (or index.js)
requirements.txt / package.json
Dockerfile
docker-compose.yml (optional)
.github/workflows/ci.yml
README.md

Minimum CI workflow (GitHub Actions):

- Trigger: on push and pull_request.
- Steps:
 - Checkout code
 - Set up runtime (Node/Python)
 - Install dependencies
 - Run tests (even a trivial test)
 - Build Docker image

This single project shows:

- Docker fundamentals
- Docker Compose (if used)
- Git discipline
- CI basics (GitHub Actions)

Daily Workflow (60 Minutes) 1. 10 min — Review yesterday’s commits + LOG.md
2. 30 min — Build/Break/Fix exercise (basics or project work)
3. 15 min — Validate, document, commit, push
4. 5 min — Update LOG.md with what you did

7-Day Structured Docker Basics Plan **Day 1 — Build Your First Dockerfile**

- Create basics/day1-basic-image/ with your own Dockerfile (no copy-paste).
- Build & run image.
- Exercise: Break the build (wrong path) and fix it.
- Commit: "Day 1: Basic Dockerfile and first image"

Day 2 — Container Lifecycle

- Folder: basics/day2-containers/
- Run containers with -p, -v, -e.
- Exercise: Make a container fail to start → use docker logs to fix.

- Commit: "Day 2: Container run, ports, env, logs"

Day 3 — Images Deep Dive

- Folder: basics/day3-images/
- Use docker image inspect, history, tags.
- Exercise: Reduce image size using Alpine or multi-stage builds.
- Commit: "Day 3: Image inspect + size optimization"

Day 4 — Volumes & Persistence

- Folder: basics/day4-volumes/
- Test bind mounts vs named volumes.
- Exercise: Data persists after container deletion.
- Commit: "Day 4: Data persistence with volumes"

Day 5 — Networking

- Folder: basics/day5-networking/
- Create a user-defined network, run two containers on it.
- Exercise: Make them talk to each other (ping / curl by container name).
- Commit: "Day 5: Custom network and container-to-container comms"

Day 6 — Docker Compose

- Folder: basics/day6-compose/
- Build a Web + DB stack with docker-compose.yml.
- Exercise: Break one service config → fix, re-run.
- Commit: "Day 6: Web + DB stack with Compose"

Day 7 — Incident Simulation

- Folder: basics/day7-incident/
- Create your own "incident": wrong image tag, missing env, broken volume, etc.
- Use logs + inspect to fix it.
- Document root cause in a small INCIDENT.md.
- Commit: "Day 7: Incident simulation and resolution"

Mini CI/CD Project — Suggested 7+ Extra Days Not tied to a calendar, but roughly:

- **Phase 1:** Simple app + Dockerfile.
- **Phase 2:** docker-compose with a second service.
- **Phase 3:** Add GitHub Actions CI (build + tests).
- **Phase 4:** Polish README with diagrams and usage instructions.

Each phase = multiple 1h sessions with separate commits and possibly branches like:

feature/app
feature/dockerfile
feature/compose
feature/ci-pipeline

Recommended Learning Sources Udemy

- <https://www.udemy.com/course/docker-mastery/>
- <https://www.udemy.com/course/docker-and-kubernetes-the-complete-guide/>

O'Reilly

- <https://learning.oreilly.com/library/view/docker-up-and/9781491917572/>
- <https://learning.oreilly.com/videos/getting-started-with/9781491988886/>

YouTube

- TechWorld with Nana — <https://www.youtube.com/watch?v=3c-iBn73dDE>
- Fireship — <https://www.youtube.com/watch?v=gAkWW2tulqE>

Free E-learning

- Official Docker training: <https://docs.docker.com/get-started/>
- Scenario-based labs (or similar platforms that provide interactive terminals) for Docker.

Exercise Answer Patterns **Day 1** → Fix build context and paths; missing file must exist and be inside context.

Day 2 → Use docker logs, fix wrong port/env var/command.

Day 3 → Use smaller base image, remove unnecessary layers, multi-stage builds.

Day 4 → Named volumes and bind mounts; verify files remain after container removal.

Day 5 → Custom network + container names instead of IP; test with ping/curl.

Day 6 → Correct service name, image tags, depends_on, env, then docker compose up -d.

Day 7 → Always: logs → inspect → config check → rebuild → redeploy; document root cause.